

PROYECTO INTEGRADO



Active Spain

DOCUMENTACIÓN

Desarrollado por: Alejandro Pintado López

2º DESARROLLO DE APLICACIONES WEB

IES Velázquez Curso 2024/2025

Para acceder a la web del proyecto:



<https://activespain.infinityfree.me/>

Importante: Esta documentación no recoge una explicación detallada del código, ya que este viene debidamente comentado dentro del mismo, en el archivo .zip adjunto a la entrega.

1) Reunión inicial

Este proyecto integrado tiene como objetivo crear una aplicación web llamada ActiveSpain que ponga en contacto OFERTANTES (quienes publican actividades: rutas, aventura, yincanas, etc) y CONSUMIDORES (personas que contratan o solicitan actividades). Incluye gestión de perfiles, publicación o búsqueda de ofertas y demandas y puesta en contacto (reserva/contacto).

El público al que va dirigido la aplicación es a particulares y pequeñas empresas/guías de ocio en España (usuarios con móvil u ordenador). Prioriza una experiencia de usuario sencilla y búsqueda geolocalizada básica.

Alcance mínimo:

- CRUD ofertantes.
- CRUD consumidores.
- Interfaz front-end (HTML/CSS/JS/Angular) y back-end (modelos y servicios en PHP).
- Base de datos relacional (MySQL).
- Documentación básica (UML, modelo ER, manual de usuario).
- Deploy en GitHub + hosting gratuito en InfinityFree (mínimo: despliegue funcional en un servidor o servicio web).

2) Requerimientos básicos

Ordenadas por prioridad, cada funcionalidad incluye su criterio de aceptación:

1. Registro / Login – roles
Como usuario quiero registrarme e iniciar sesión con rol ofertante o consumidor.
Criterio de aceptación: registro con email + contraseña + rol, login con JWT, protección de rutas por rol, contraseña protegida con password_hash, longitud mínima, validación de email.
2. CRUD Ofertantes
Como ofertante quiero crear, editar, ver o borrar mi perfil y mis actividades.
Criterio de aceptación: endpoints con GET /ofertas, POST /ofertas, PUT /ofertas/:id, DELETE /ofertas/:id; sólo el creador puede editar o borrar su oferta; validaciones (titulo, tipo, lugar, fecha y hora de inicio) y manejo de errores con códigos 4xx/5xx.
3. CRUD Consumidores

Como consumidor quiero crear, editar, ver o borrar mi perfil y mis solicitudes o demandas.

Criterio de aceptación: endpoints análogos a ofertas, con validaciones (título, lugar, fecha y hora deseada). Sólo el creador puede modificar o borrar su consumo.

4. Listado y búsqueda básica de actividades y peticiones

Como usuario, independientemente de mi rol, quiero ver y buscar actividades o peticiones por categoría, palabras clave y ciudad.

Criterio de aceptación: lista paginada, filtros por tipo, búsqueda por texto, mostrar disponibilidad mínima.

5. Protección de rutas

Lista de rutas que requieren autenticación/rol y el comportamiento esperado.

6. Interfaz agradable

Como usuario quiero una UI limpia y responsiva.

Criterio de aceptación: CSS aplicado; navegación clara; responsive en móvil/ordenador.

7. Publicar demanda por consumidor

Permite a consumidores publicar solicitudes que ofertantes vean.

8. Disponibilidad y plazas

Ofertantes establecen fecha y hora, plazas mínimas y máximas.

9. Material y transporte

Campos para indicar material necesario/ofertado y transporte requerido.

3) Diseño de la arquitectura mínima

Componentes principales y herramientas utilizadas:

- Frontend: Angular (standalone con Single Page Application).
- Backend: PHP “vanilla” (scripts RESTful o front controller) que exponen JSON.
- Base de Datos: MySQL en el servidor de InfinityFree.
- Hosting para desarrollo: XAMPP — coloca el backend en C:\xampp\htdocs\Ejercicios\ProyectoIntegrado\Backend y el frontend en C:\xampp\htdocs\Ejercicios\ProyectoIntegrado\Frontend y utiliza ng serve localmente.
- Comunicación: HTTPS idealmente (en producción), JSON sobre HTTP(S).
- UML: PlantUML
- Diagrama E/R: Draw.io

- Deploy: Realizamos `ng build --configuration production` y los archivos resultantes de la carpeta 'dist' dentro del front-end, junto a los ficheros PHP del back-end son subidos con FileZilla al servicio de hosting con InfinityFree.

4) Planificación de tareas

Sprint 1 (10 horas) – Base del Proyecto

Configuración del entorno (repositorio, frameworks, librerías).

Implementación del back-end básico (API REST, modelo de datos).

Implementación del front-end básico (estructura y estilos iniciales).

Pruebas unitarias y ajuste de errores iniciales.

Sprint 2 (10 horas) – Funcionalidades Principales

Desarrollo de funciones clave (CRUD, autenticación si aplica).

Integración de front-end y back-end.

Optimización del código y pruebas.

Revisión intermedia y ajustes.

Sprint 3 (10 horas) – Mejoras y Ajustes Finales

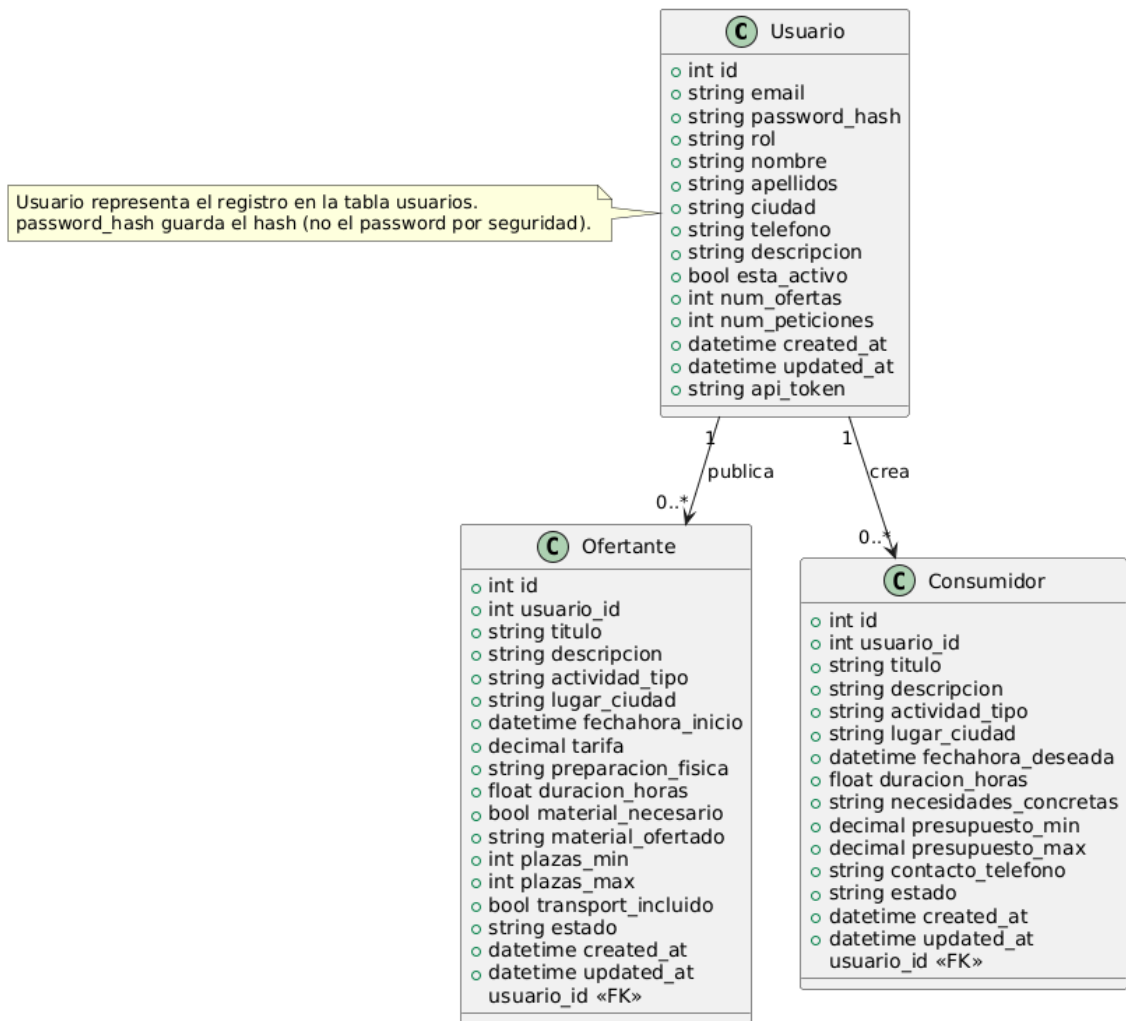
Corrección de errores y refactorización.

Implementación de mejoras (seguridad, UI/UX, rendimiento).

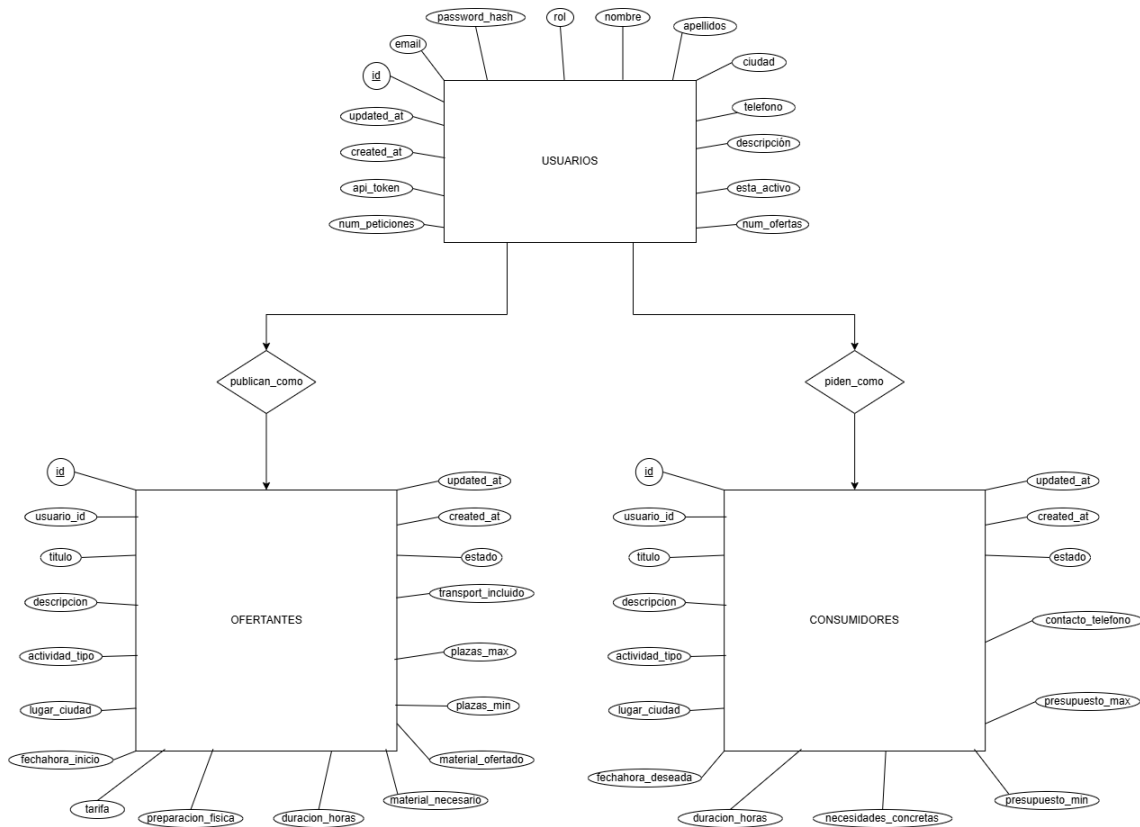
Pruebas finales (manuales y automatizadas).

Preparación para entrega (documentación, despliegue si aplica).

5) Diagrama Lenguaje Unificado de Modelado (UML)



6) Diagrama Entidad-Relación



7) Elecciones de diseño y tecnología — por qué y qué me aportaron

Frontend: Angular (Single Page Application)

- **Por qué lo escogí:** Angular proporciona una estructura robusta para proyectos medianos, binding (ligadura) y formularios reactivos (FormGroup) que facilitan validaciones y gestión de estado de formularios (ofertas/peticiones).
- **Qué me aportó:** Desarrollo rápido de componentes reutilizables (modales, formularios), manejo limpio de formularios complejos (validaciones, getRawValue), y una buena separación entre vista y servicios.
- **Coste/complicación:** Requiere build (dist) y configuración de deployment; si el dist no está donde espera el host, la web no aparece (404 en Netlify si la carpeta publicada no existe). Además, hay que cuidar la configuración de angular.json (outputPath) y las rutas base si se publica en subdirectorío.

Backend: PHP (modelos.php + servicios.php)

- **Por qué lo escogí:** Ligero, sencillo de desplegar en hosting compartido y fácil de integrar con MySQL. Me permitió implementar rápidamente endpoints REST que devuelven/reciben JSON.
- **Qué me aportó:** Control completo sobre consultas, validaciones y generación de api_token (login). Fácil de depurar en local con XAMPP.
- **Coste/complicación:** Seguridad y gestión de credenciales (no meter usuario/contraseña de BD en GitHub), y problemas con headers HTTP (CORS/Authorization) según el entorno de hosting.

Base de datos: MySQL (InfinityFree)

- **Por qué lo escogí:** Sencillo y compatible con PHP; InfinityFree ofrece MySQL en gratuito lo que facilita hacer pruebas en remoto.
- **Qué me aportó:** Tener una base de datos pública para el despliegue final; pude comprobar que las tablas usuarios, ofertas, peticiones estaban creadas y accesibles.
- **Coste/complicación:** Limitaciones del hosting (restricciones de acceso remoto, reglas de seguridad, inyección de scripts de protección), y la necesidad de adaptar PDO DSN y permisos.

8) Dificultades relevantes encontradas y cómo las abordé (detalle técnico)

A) Conexiones a la base de datos y permisos (error 1044)

- **Situación:** Error Access denied for user 'if0_40609679'@'192.168.%' to database 'proyecto_integrado'. Esto ocurre cuando el usuario de MySQL no tiene permisos sobre la base de datos o se usa el usuario equivocado.
- **Lección:** En hostings compartidos a menudo el prefijo del nombre de BD y usuario cambian (if0_xxx_nombre). Verificar en panel de control exactos DB name, DB user, DB host y actualizar modelos.php.
- **Solución aplicada/propuesta:** Confirmar en panel de InfinityFree que el usuario tiene permiso a la BD (en panel de hosting se crea la BD y se asigna el usuario); actualizar pdo DSN en modelos.php. Evitar exponer estas credenciales en repositorios públicos.

B) Autenticación: el token parece existir pero el backend dice “No autenticado”

- **Síntomas observados:** En el frontend se envía Authorization: Bearer <token> y en application > cookies aparece __test (cookie automática de InfinityFree/anti-bot). Sin embargo, en servicios.php getBearerToken() no detectaba el header y authUser quedaba null.
- **Causas y matices:**
 - Algunos entornos (cient hosting, proxies o servidores) no pasan la cabecera Authorization por defecto a PHP (Apache a veces no la expone en \$_SERVER sin un ajuste).
 - InfinityFree/Cloudflare y servicios similares pueden inyectar un JS anti-bot que establece cookies y hasta redirige a una URL temporal mientras valida el visitante (ese __test y el aes.js que viste).
 - Si usas Access-Control-Allow-Origin: * y además necesitas enviar credenciales (cookies), esto no funcionará: * + Access-Control-Allow-Credentials: true es inválido. Hay que decidir: token en Authorization header (mejor) o cookie HttpOnly.
- **Qué hice/probé:**
 - Implementé getBearerToken() que intenta leer apache_request_headers() y también \$_SERVER['HTTP_AUTHORIZATION'] como fallback.
 - Añadí lógica en el frontend para adjuntar token tanto en header Authorization como en el body (attachTokenToBody) como plan B — esto ayudó con hosts que eliminan la cabecera.
 - Creé un endpoint debug (servicios_debug.php) para inspeccionar qué headers y body llegan al servidor. Con esto confirmé que el token **sí** llegaba en el body en pruebas, y que la DB devolvía usuario correcto cuando el token se buscaba en la BD.
- **Recomendaciones finales:**
 - Preferir Authorization: Bearer en headers; en servidores que lo bloquean, habilitar RewriteRule o SetEnvIf Authorization "(.*)" HTTP_AUTHORIZATION=\$1 en .htaccess para pasar la cabecera a PHP.
 - Si decides usar cookies, ponerlas HttpOnly y seguras, y configurar CORS con Access-Control-Allow-Credentials: true y un origen concreto (no *).

- Evitar confiar en soluciones “token en body” a la larga; es menos estándar.

C) Cookies anti-bot inyectadas por el hosting (aes.js, __test)

- **Situación:** Al cargar el endpoint remoto, recibí un HTML con un script aes.js que crea __test cookie y redirige a la URL con i=1. Eso es una protección anti-bot que el hosting gratis (o Cloudflare intermedio) añade.
- **Impacto:** Añade complejidad para debugging y autenticación (redirecciones, headers modificados). A veces el script impide que el Authorization llegue limpio.
- **Cómo lo gestioné:** Usé el endpoint de debug y observé que, tras la validación, la petición correcta llega y el token es detectado desde el body. En producción, si quieres evitarlo, necesitas un plan de hosting que no inyecte esas protecciones o configurar headers/paths que el proveedor permita sin retoques.

D) Caching / persistencia de la versión local vs remota

- **Síntoma observado:** Aunque borré el dist local y cambié archivos, el frontend seguía mostrando la versión anterior (como “guardada”), y borrar caché del navegador no parece solucionar.
- **Causas posibles rastreadas:**
 - Service Worker: la app podría haber registrado un service worker que cachea assets. Si existe, es necesario actualizar o desactivar el SW en producción.
 - CDN / proxy cache en el hosting o en el navegador.
 - Archivos dist en el servidor no fueron actualizados; el servidor seguía sirviendo la versión antigua.
- **Solución/propuesta:** Comprobar navigator.serviceWorker.getRegistrations() en consola para ver si hay SW activos; si existen, desregistrarlos durante el despliegue o actualizar versiones de SW. Forzar Cache-Control apropiado en el servidor.

9) Decisiones funcionales y de diseño que considero bien tomadas (y por qué)

- **Tokens per-user en DB (api_token):** simple y suficiente para un MVP. Permite invalidar tokens desde la BD y no requiere un sistema de JWT complejo si la escala es pequeña.

- **Separación FRONT / BACK con JSON:** modela bien el flujo SPA y permite migrar backend a otra tecnología si lo necesito.
- **Validaciones mínimas en backend (modelo):** protegen contra inserciones mal formadas y previenen duplicados (email único). Mantener lógica crucial en backend siempre.
- **Debug endpoints temporales:** Crear un endpoint de depuración fue clave para comprobar qué llegaba realmente al servidor y localizar el problema del Authorization.
- **Fallback de token en body (attachTokenToBody):** no es ideal, pero me permitió mantener la app operativa con un hosting que elimina cabeceras Authorization.

10) Aspectos de seguridad y buenas prácticas que detecté y aplicaría / mejoraría

- **No exponer credenciales en el repositorio:** las credenciales de BD no deben subirse. Usar variables de entorno o ficheros fuera del control de versiones.
- **Tokens en localStorage vs cookies:** guardar token en localStorage es cómodo, pero no protege contra XSS. Lo ideal en producción es HttpOnly cookie con CSRF token para peticiones cambiantes. Para el alcance del proyecto un api_token en localStorage es aceptable, con la salvedad de endurecer sanitización de la UI para minimizar XSS.
- **CORS con credenciales:** si se usa cookie HttpOnly, hay que configurar Access-Control-Allow-Credentials: true y Access-Control-Allow-Origin a un origen específico, **no ***.
- **Rate limiting / hardening:** un hosting gratuito no ofrece mucho control, así que recomiendo un plan con más controles para un producto real.

11) Reflexión personal

He disfrutado mucho el proceso técnico y la parte creativa del proyecto — montar la UI en Angular y luego conectar todo con PHP fue un flujo natural para este tipo de aplicaciones. Lo más frustrante fue lidiar con los aspectos prácticos del hosting compartido (cabeceras eliminadas, scripts anti-bot, caching inesperado), porque eran errores que no provenían del código sino del entorno; esos problemas consumen mucho tiempo porque obligan a investigar a fondo y probar varias alternativas (headers, body, debug endpoints, .htaccess).

Mejoré también la resiliencia del proyecto: añadí comprobaciones adicionales, debug endpoints y fallbacks (token en body) para que la app siguiera funcionando bajo condiciones imperfectas. Aun así, para una versión más profesional, recomiendo migrar a un hosting con mayor control o a un entorno donde yo controle completamente los headers y las cabeceras HTTP.